AWS SERIES · MODULE 4 OF 8

Amazon DynamoDB: Fast Data by Key

A beginner's guide to Amazon DynamoDB: how a NoSQL table stores items, why the key you choose decides everything, how to read and write safely, and how a GenAI assistant uses DynamoDB for chat memory — with hands-on steps in the console and on the command line.

PA

Tutor: Prudhvi Akella

Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Services: Amazon DynamoDB · Partition & sort keys · Global Secondary Indexes · Conditional writes · TTL · Streams

Before this module: Module 1 (account and CLI set-up) and Module 2 (IAM). Region: `us-east-1`. Sample: AcmeCorp HR assistant — employee records and chat history

1. What DynamoDB is
2. Tables, items and attributes
3. Primary key 1: partition key only
4. Primary key 2: partition key + sort key
5. How DynamoDB spreads data
6. Data types and "DynamoDB JSON"
7. Writing: Put, Update, Delete
8. Reading: GetItem, Query, Scan
9. Querying with the sort key
10. Filters and pages
11. Design from your questions
12. Secondary indexes (GSI)
13. Relational database vs DynamoDB: side by side
14. SQL index vs Global Secondary Index
15. The same questions in SQL and in DynamoDB
16. Related data without joins
17. Capacity: on-demand vs provisioned
18. Consistency
19. Safe updates: conditions and counters
20. TTL and Streams
21. DynamoDB in a GenAI app: chat memory
22. DynamoDB, RDS or S3?
23. What DynamoDB costs
24. What you can skip for now
25. Troubleshooting
26. Build it yourself: what you will build
27. Before you start
28. Build 1 · Create the Employees table
29. Build 2 · Add employees
30. Build 3 · Read: get and scan
31. Build 4 · Update safely
32. Build 5 · Find by department (GSI)
33. Build 6 · Chat history table with TTL
34. Build 7 · Clean up (same day)
35. Key takeaways

1 What DynamoDB is

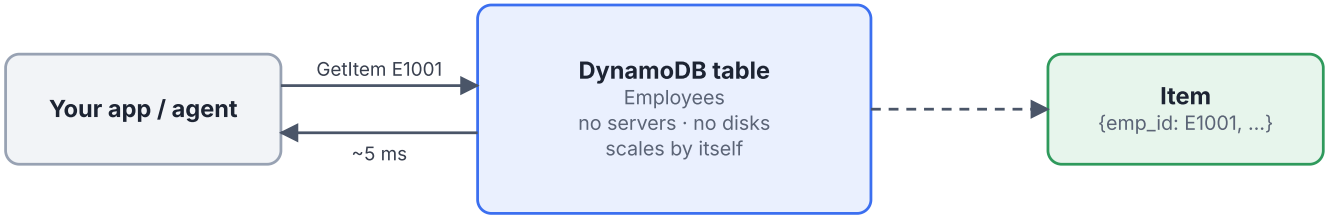


Figure 1. You give DynamoDB a key; it gives back the item in a few milliseconds — at any size, with no servers to run.

Amazon DynamoDB is a fully managed **NoSQL** database. You create a table and start reading and writing. There are no servers, no software versions and no disks to manage. Reads and writes by key take a few milliseconds, whether the table has 10 items or 10 billion.

	SQL database (e.g. RDS PostgreSQL)	DynamoDB
Shape	Fixed columns for every row	Only the key is fixed; each item can have different attributes
How you ask	Any SQL query, joins across tables	Mainly **by key**; no joins
Scaling	Bigger server, read replicas	Automatic, no limit you will hit
You manage	Instance size, patches, storage	Almost nothing

> The trade-off: DynamoDB is very fast and simple **if you look things up by key**. If you need free-form questions and joins, use a SQL database. The full comparison is in sections 13–16.

2 Tables, items and attributes

Table: Employees

emp_id (key)	name	department	city	leave_balance	skills
E1001	Anita	HR	Hyderabad	18	–
E1002	Ravi	Finance	Mumbai	12	–
E1003	Sana	IT	Pune	20	python, aws

Figure 2. Table Employees. Every item has the key emp_id; the other attributes can differ per item.

DynamoDB	Like in SQL	Note
Table	Table	Lives in one Region
Item	Row	Max 400 KB per item
Attribute	Column	A name and a value. Items can have different attributes

DynamoDB	Like in SQL	Note
Primary key	Primary key	The only required attribute. Unique per item

The same item, written as normal JSON:

```
{ "emp_id": "E1003", "name": "Sana", "department": "IT", "city": "Pune",
  "leave_balance": 20, "skills": ["python", "aws"] }
```

Only `emp_id` was declared when the table was created. `name`, `city`, `skills` were never declared — you just write them. That is what "schemaless" means.

3 Primary key 1: partition key only

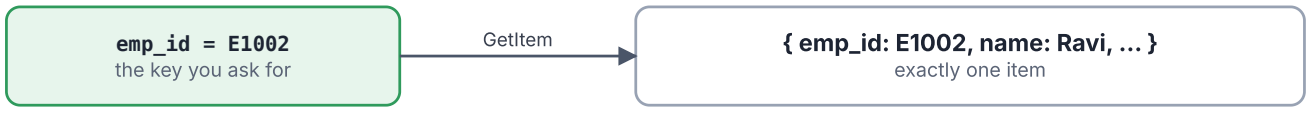


Figure 3. Partition key only: one key value → one item.

The simplest key is a single attribute called the **partition key**. Every item must have a different value. You use it when you always fetch **one thing by its ID**: an employee, a user profile, a document's status.

Table	Partition key	Typical question
Employees	`emp_id`	"Give me employee E1002"
Documents	`doc_id`	"Has leave-policy.pdf been embedded yet?"
Users	`user_id`	"What is this user's preferred language?"

4 Primary key 2: partition key + sort key



Figure 4. Partition key + sort key: one session holds many messages, kept sorted by time.

A **composite key** has two parts. The **partition key** groups related items together. The **sort key** orders the items **inside** that group. Together they must be unique.

Table	Partition key	Sort key	Question it answers in one call
ChatMessages	`session_id`	`created_at`	"Last 10 messages of session S-100"
LeaveRequests	`emp_id`	`request_date`	"Anita's leave requests in 2026"
DocChunks	`doc_id`	`chunk_no`	"All chunks of leave-policy.pdf, in order"

Sort keys are compared as text or numbers. Store times as ISO text (`2026-11-01T09:00:01Z`) or as numbers — both sort correctly, so "newest first" and "between two dates" just work.

5 How DynamoDB spreads data

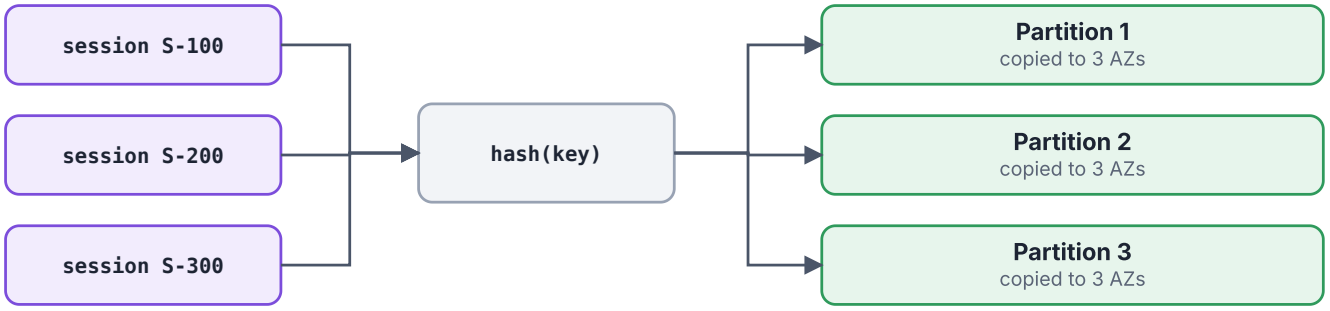


Figure 5. The partition key is hashed; the hash decides which partition (storage node) holds the items.

DynamoDB splits a table into **partitions** and spreads them over many machines. The partition key decides where an item lives. Each partition handles up to about **3,000 reads and 1,000 writes per second**, and DynamoDB adds partitions as the table grows.

Partition key choice	Result
Many different values, used evenly (`session_id` , `emp_id` , `doc_id`)	Load spreads over many partitions → fast at any scale
Few values used by everyone (`status = NEW` , `country = IN` , today's date)	All traffic hits one partition → **throttling** (a "hot key")

Rule: pick a partition key with **many distinct values** that requests are **spread across**.

6 Data types and "DynamoDB JSON"

Type code	Type	Example value
`S`	String	`"Anita"`
`N`	Number (sent as text)	`"18"`
`BOOL`	true / false	`true`
`NULL`	Empty	`true`

Type code	Type	Example value
`M`	Map (an object)	`{"role": {"S": "user"}}`
`L`	List	`[{"S": "a"}, {"N": "1"}]`
`SS` / `NS`	Set of strings / numbers	`["python", "aws"]`

The CLI and the low-level API use **DynamoDB JSON**, where every value is wrapped with its type code. It looks heavy, but it is the same data:

Normal JSON	DynamoDB JSON (what the CLI wants)
`{"emp_id": "E1001", "leave_balance": 18}`	`{"emp_id": {"S": "E1001"}, "leave_balance": {"N": "18"}}`

> You only see DynamoDB JSON in the **CLI** and the raw API. The **console** lets you switch it off, and Python's `boto3.resource("dynamodb")` (used in the notebook) takes normal Python values.

7 Writing: Put, Update, Delete

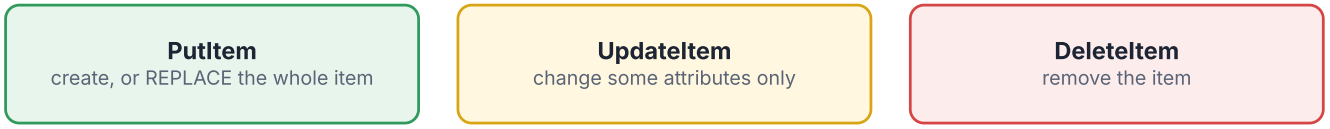


Figure 6. The three write calls. All of them need the full primary key.

Call	Does	Watch out
`PutItem`	Writes a whole item	If the key already exists, the old item is **replaced completely** — attributes you left out are gone
`UpdateItem`	Changes listed attributes (`SET` , `REMOVE` , `ADD`)	Creates the item if it does not exist
`DeleteItem`	Removes one item by key	No error if it was not there
`BatchWriteItem`	Up to 25 puts/deletes in one call	Each item succeeds or fails on its own — check `UnprocessedItems`

```
UpdateExpression:  SET leave_balance = leave_balance - :days, updated_by = :who
Values:            :days = 2,  :who = "anita"
Result:            leave_balance 18 → 16, everything else untouched
```

8 Reading: GetItem, Query, Scan

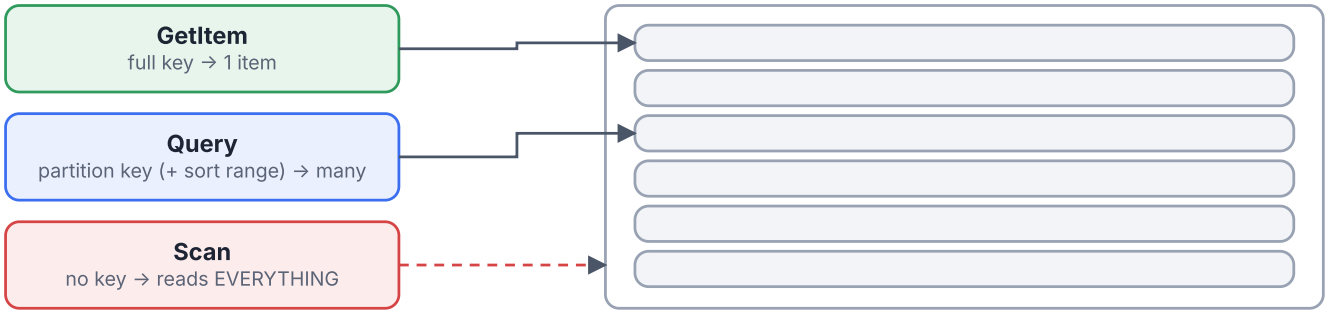


Figure 7. GetItem and Query go straight to the data. Scan reads the whole table.

Call	You give	Reads	Use it for
`GetItem`	The full primary key	1 item	Fetch one known thing
`Query`	Partition key (+ optional sort key condition)	Only matching items	All messages of a session, a person's requests
`Scan`	Nothing	The **whole table**	One-off exports or tiny tables — not in app code
`BatchGetItem`	Up to 100 keys	Those items	Fetch a known list of IDs

If your app needs a `Scan` to answer a question every time, the key design is wrong for that question. Fix the key or add an index (section 12).

9 Querying with the sort key

A `Query` always has **partition key = value**. You can then narrow the sort key:

Sort key condition	Example on ChatMessages	Returns
(none)	`session_id = S-100`	All messages of the session, oldest first
`>` `<` `>=` `<=`	`created_at > 2026-11-01T09:01`	Messages after 09:01
`BETWEEN … AND …`	`created_at BETWEEN 09:00 AND 09:30`	Messages in that window
`begins_with`	`begins_with(created_at, 2026-11-01)`	Messages on 1 Nov

Option	Effect
`ScanIndexForward = false` (CLI: `--no-scan-index-forward`)	Newest first
`Limit = 10`	Stop after 10 items
Both together	**The last 10 messages** — exactly what a chat assistant loads as memory

10 Filters and pages

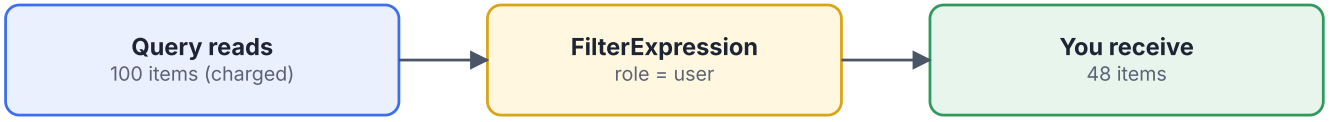


Figure 8. A filter runs AFTER the read. You pay for everything read, not only what is returned.

- A **filter** removes items from the result **after** they are read. It saves network data, not cost or time.
- One `Query` or `Scan` call returns at most **1 MB**. If there is more, the response includes `LastEvaluatedKey`. Call again with it (`ExclusiveStartKey`) to get the next page. The CLI does this for you automatically.
- Count without fetching items: `Select = COUNT`.

11 Design from your questions

In SQL you design tables first and write queries later. In DynamoDB you do the **opposite**: write down every question the app will ask, then choose keys so each question is one `GetItem` or `Query`.

#	Question the HR assistant asks	Answered by
1	Get employee E1001	`GetItem` on Employees, key `emp_id`
2	Last 10 messages of a chat session	`Query` on ChatMessages, `session_id`, newest first, limit 10
3	Save a new message	`PutItem` on ChatMessages
4	All employees in the HR department	`Query` on a GSI with key `department` (next section)
5	Delete chats older than 30 days	TTL attribute `expires_at` (section 20)

> Do this table **before** creating anything. If a question would need a `Scan`, change the key or add an index now.

12 Secondary indexes (GSI)



Figure 9. A GSI is a second copy of the table, sorted by a different key. DynamoDB keeps it updated for you.

The table can only be queried by its own key. To ask by another attribute — e.g. **department** — add a **Global Secondary Index (GSI)**. You then `Query` the index instead of the table.

- Add a GSI any time, also to a table that already has data.
- Reads from a GSI are eventually consistent (a write shows up in the index a moment later).
- Every write to the table is also written to each GSI that includes it, so each GSI adds write cost.
- Up to 20 GSIs per table by default — most tables need 0 to 3.

13 Relational database vs DynamoDB: side by side

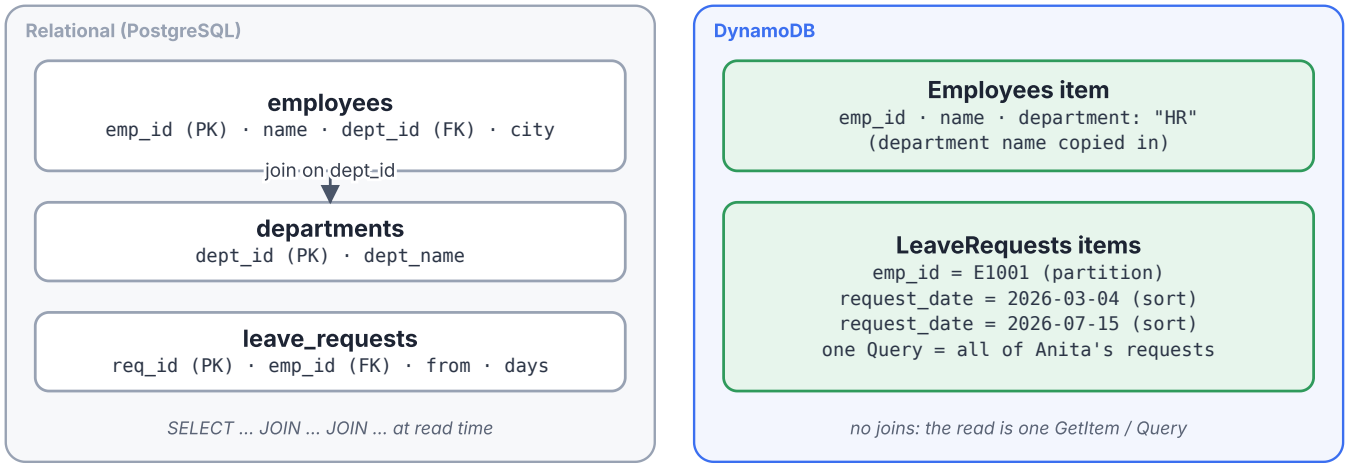


Figure 10. The same HR data. SQL splits it into tables and joins at read time; DynamoDB stores it the way it is read.

A relational database (PostgreSQL, MySQL, RDS, Aurora) **splits data into many tables** so each fact is stored once, then **joins** them when you read. DynamoDB does the opposite: it **stores data shaped the way you will read it**, so every read is one simple call by key.

Concept	Relational database	DynamoDB
Table / row / column	Table / row / column	Table / item / attribute
Schema	Every column declared up front; `ALTER TABLE` to change	Only the key declared; other attributes free per item
Primary key	Any column(s); often an auto-increment number	Partition key, or partition + sort key. No auto-increment — you choose IDs
Relationships	Foreign keys, enforced by the database	None enforced. You store IDs or copy values in
Joins	Yes, any tables, at read time	**No.** Shape data for the read, or make two calls
Extra ways to look up	`CREATE INDEX` on any column(s)	**GSI** (any new key) or LSI (new sort key) — next section
Ad-hoc questions	Any `SELECT … WHERE … GROUP BY`	Only what your keys and indexes support; otherwise `Scan`
Aggregates (SUM, COUNT … GROUP BY)	Built in	Not built in — compute in code, or keep counters updated on write

Concept	Relational database	DynamoDB
Transactions	Any number of rows, any tables	Up to 100 items per transaction
Scaling	Bigger server, read replicas; writes go to one primary	Spread over partitions automatically; reads and writes both scale
Data design	**Normalise**: store each fact once	**Denormalise**: copy data where it is read; design from the questions
Speed at huge size	Depends on query plan, indexes, server size	Same few milliseconds by key at any size

> Neither is "better". Relational = flexible questions on related data. DynamoDB = known questions, by key, at any scale with no servers.

14 SQL index vs Global Secondary Index

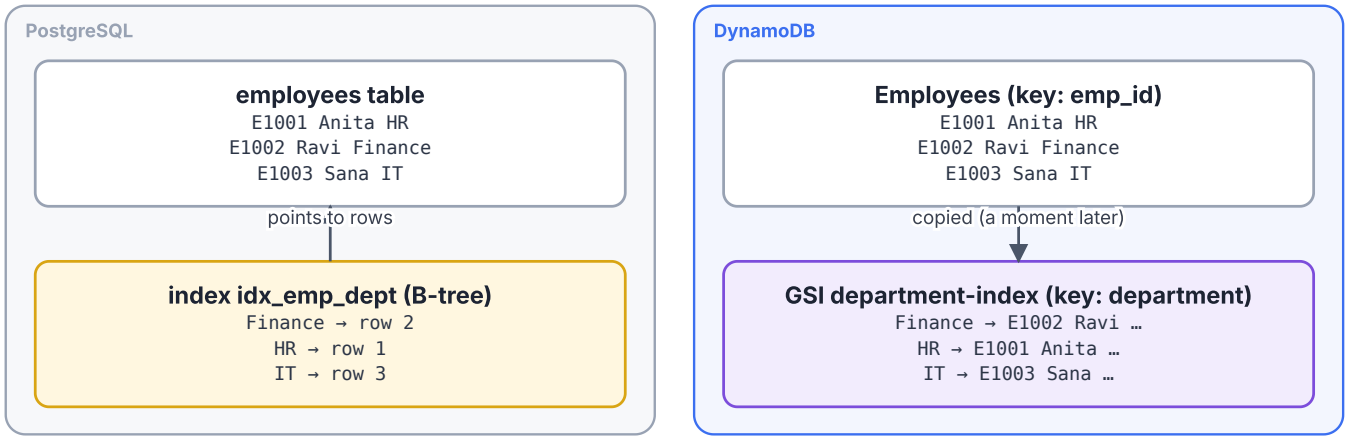


Figure 11. A SQL index points back into the same table. A GSI is a separate, automatically updated copy with its own key.

Both let you find rows/items by something other than the primary key. They work differently:

	SQL index (`CREATE INDEX`)	DynamoDB GSI
What it is	A sorted lookup structure **pointing to rows** in the same table	A **separate copy** of the items, stored under a **different key**
How you use it	Just write `WHERE department = 'HR'` ; the database picks the index	You must **name it**: `Query` with `IndexName = department-index`
Up to date?	Updated in the **same transaction** as the row	Updated **a moment later** (eventually consistent); no strong reads
Unique?	Can enforce `UNIQUE`	**Never unique** — many items can share a GSI key (all HR employees)
Which columns	Index stores the key; row data fetched from the table	You choose what to copy: **keys only**, **some attributes**, or **all**
Items missing the attribute	Row still in index (as NULL)	**Left out of the GSI** (a "sparse" index — useful, e.g. only items with `status`)

	SQL index (`CREATE INDEX`)	DynamoDB GSI
Key shape	One or many columns, any order	Partition key (+ optional sort key) — same rules as a table
Can query	`=` , ranges, `LIKE 'abc%'` , combinations	Partition key `=` , sort key ranges / `begins_with` only
Cost	Some extra write work and disk	Extra **write charge** on every table write that touches it, plus storage
When to add	Any time	Any time (an **LSI** must be created with the table)
Limit	Practically many	20 GSIs + 5 LSIs per table by default

GSI vs LSI in one line each

Index	Key	Like in SQL
GSI (Global Secondary Index)	A completely new partition key (+ sort key)	An index on another column, e.g. `(department)` or `(department, city)`
LSI (Local Secondary Index)	Same partition key, a **different sort key**; only at table creation	A composite index `(emp_id, status)` next to the primary `(emp_id, request_date)`

> Rule of thumb from SQL: if you would write `CREATE INDEX … ON (x)` because you query `WHERE x = …` , the DynamoDB answer is usually a **GSI with partition key x**.

15 The same questions in SQL and in DynamoDB

```
-- PostgreSQL: tables and index
CREATE TABLE employees (emp_id TEXT PRIMARY KEY, name TEXT, department TEXT, city TEXT,
leave_balance INT);
CREATE INDEX idx_emp_dept ON employees (department);

# DynamoDB: table with key emp_id, plus a GSI on department
aws dynamodb create-table --table-name Employees ... --key-schema
"AttributeName=emp_id,KeyType=HASH"
aws dynamodb update-table --table-name Employees ... --global-secondary-index-updates
file://gsi.json
```

Question	SQL	DynamoDB	Good fit?
One employee	`SELECT * FROM employees WHERE emp_id = 'E1001'`	`GetItem` key `emp_id`	✓ both
All HR staff	`… WHERE department = 'HR'` (uses the index)	`Query` GSI `department-index`	✓ both

Question	SQL	DynamoDB	Good fit?
Anita's leave in 2026	`… FROM leave_requests WHERE emp_id='E1001' AND from_date >= '2026-01-01'`	`Query` LeaveRequests, `emp_id` + sort key `>=`	✓ both
Take 2 leave days if enough	`UPDATE … SET leave_balance = leave_balance-2 WHERE emp_id='E1001' AND leave_balance>=2`	`UpdateItem` + `ConditionExpression`	✓ both
Employees + department head's name	`JOIN departments`	Copy head's name into each item, or 2 calls	SQL easier
Count staff per department	`SELECT department, COUNT(*) … GROUP BY department`	No GROUP BY — keep a counter item, or Scan	SQL easier
Names containing "an"	`WHERE name ILIKE '%an%'`	Scan + filter (reads everything)	SQL / search engine
Last 10 chat messages, 1 billion rows	Works, needs care at that size	`Query` newest first, `Limit 10` — same speed at any size	DynamoDB easier

> If your list of questions is mostly the top rows, DynamoDB fits. If it is mostly the bottom rows — joins, GROUP BY, text search, "questions we haven't thought of yet" — use a relational database (or add one for reporting).

16 Related data without joins

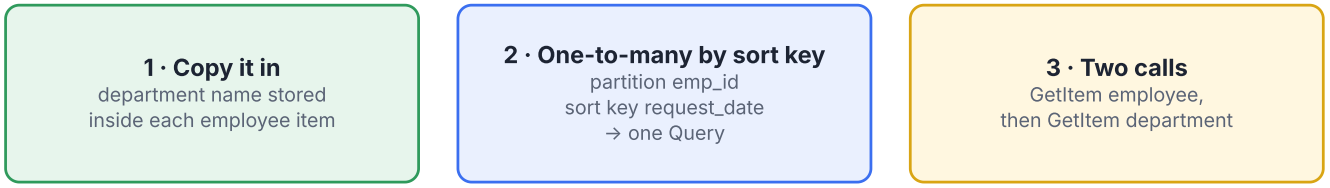


Figure 12. Three ways to handle related data without joins.

Way	How	Use when	Cost
Copy it in (denormalise)	Store `department_name` inside each Employees item	The copied value rarely changes	If it changes, update every copy
One-to-many in one table	Partition key = parent ID, sort key = child ID/date (e.g. `emp_id` + `request_date`)	You read the children of one parent together	Design the keys carefully up front

Way	How	Use when	Cost
Two calls	Read the employee, then read the department by its ID	Rare reads, small data	Two round trips (still only milliseconds)

Relational habit to unlearn: "never store anything twice". In DynamoDB, storing a small value twice to avoid a join is normal.

17 Capacity: on-demand vs provisioned

	On-demand	Provisioned
You set	Nothing	Reads and writes per second (capacity units), optionally with auto scaling
You pay	Per request	Per hour for the capacity you set, used or not
Best for	New apps, spiky or unknown traffic, labs	Steady, predictable traffic
If traffic jumps	Scales up by itself	Requests above your setting are throttled

Use **on-demand**. Move to provisioned only when traffic is steady and the bill shows a clear saving.

18 Consistency

Read type	Sees a write made a moment ago?	Cost
Eventually consistent (default)	Usually within a second	Half
Strongly consistent (`ConsistentRead = true`)	Always	Full; not available on GSIs

For chat memory, use a strongly consistent read when loading history right after saving the user's message, so the newest message is never missed.

19 Safe updates: conditions and counters

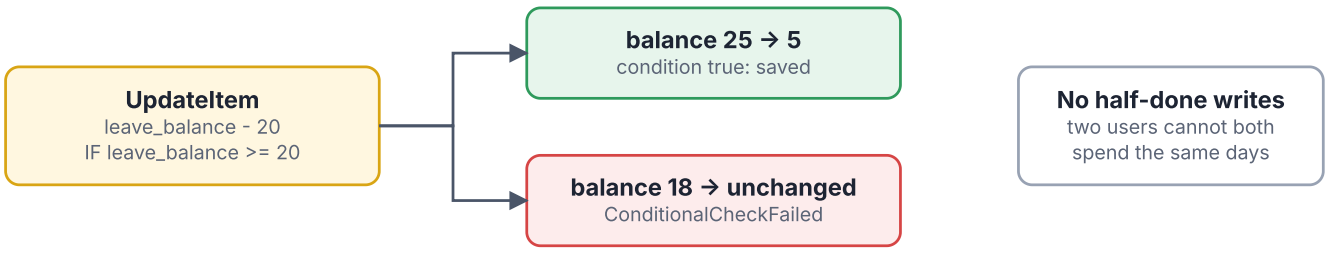


Figure 13. A condition makes the write happen only if the item is still as you expect.

Tool	Example	Protects against
Condition expression	`leave_balance >= :days`	Spending more leave than you have; two updates racing
`attribute_not_exists(emp_id)`	Only create if new	Overwriting an existing item with `PutItem`
Atomic counter	`SET views = views + :one`	Lost updates when many requests add at once
Transactions	`TransactWriteItems` (up to 100 items)	Half-finished multi-item changes (all or nothing)

20 TTL and Streams

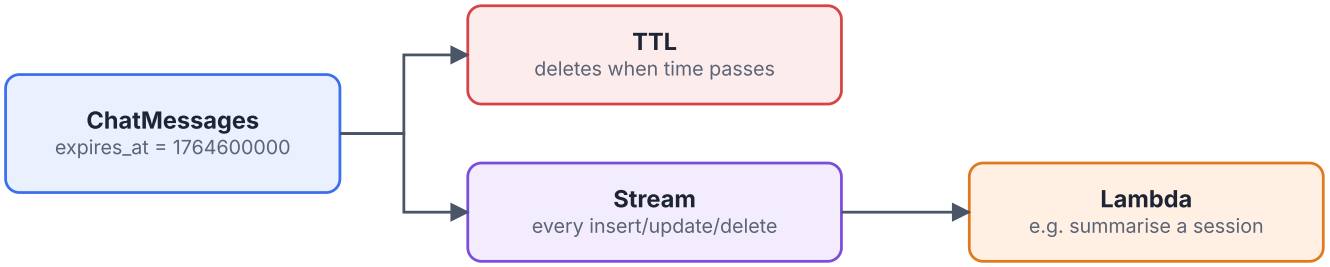


Figure 14. TTL deletes expired items for free. Streams let other code react to every change.

- **TTL (Time To Live):** choose a number attribute (e.g. `expires_at`) holding a time in **Unix seconds**. After that time, DynamoDB deletes the item **for free**, usually within a few days. Use it for chat sessions, OTPs, temporary carts, caches.
- **Streams:** a 24-hour log of every change. A Lambda can read it — e.g. to summarise a finished chat or update a search index.

> TTL is not exact to the second. If expired items must never be returned, also add a filter `expires_at > now`.

21 DynamoDB in a GenAI app: chat memory

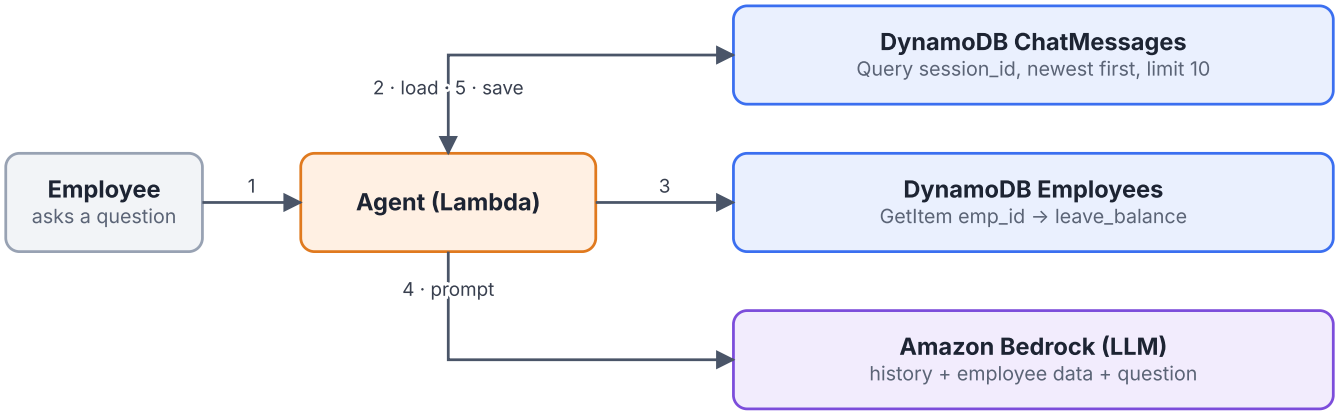


Figure 15. An HR assistant using DynamoDB as conversation memory. Numbers match the steps.

1. **An employee asks:** "Can I carry my leave over?" in session `S-100`.
2. **The agent loads memory:** `Query` ChatMessages for `S-100`, newest first, last 10 messages.
3. **It fetches facts:** `GetItem` Employees `E1001` → `leave_balance = 18`.
4. **It calls the LLM** with the chat history, the employee facts and the question.
5. **It saves both turns** (question and answer) with `PutItem`, each with `expires_at` = now + 30 days.

Other GenAI uses	Key design
Document registry for RAG ingestion (status: new / embedded / failed)	`doc_id`
Cache of LLM answers for repeated questions	`hash(prompt)` + TTL
Per-user token usage and rate limits	`user_id` + `day`, atomic counter
Agent run / tool-call log	`run_id` + `step_no`

> LangChain has a ready DynamoDB chat-history class, and agent frameworks can store their state in DynamoDB. Underneath, it is exactly this table: `session_id` + time, newest first.

22 DynamoDB, RDS or S3?

Need	Pick	Why
Look up small records by ID, very fast, any scale	**DynamoDB**	Millisecond reads by key, no servers
Reports, joins, ad-hoc SQL questions	**RDS / Aurora**	SQL can answer questions you did not plan for
Files: PDFs, images, datasets, large JSON	**S3**	Cheap per GB, any size (items in DynamoDB max 400 KB)
Semantic search over embeddings	**Vector store** (S3 Vectors, OpenSearch ...)	Built for similarity search

> Common pattern: the PDF lives in **S3**, its processing status lives in **DynamoDB**, its vectors live in a **vector store**.

23 What DynamoDB costs

You pay for	Notes
Reads and writes	On-demand: per request. Bigger items cost more per request (writes per 1 KB, reads per 4 KB)
Storage	Per GB per month. Includes indexes
Extras	Backups, Streams reads, global tables — only if you turn them on

You pay for	Notes
Always free	DynamoDB has an always-free monthly allowance (including 25 GB of storage) — labs fit easily

> Cost traps: `Scan` in app code, many GSIs on a busy table, and keeping old chat history forever (use TTL).

24 What you can skip for now

Topic	One line
Local Secondary Index (LSI)	Same partition key, different sort key; must be made with the table
Single-table design	Many entity types in one table to save calls — powerful, but advanced
Global tables	The same table copied to several Regions
DAX	An in-memory cache in front of DynamoDB
PartiQL	SQL-like syntax for DynamoDB (still by key underneath)
Point-in-time recovery, export to S3	Backups and analytics exports

25 Troubleshooting

Error	Usual cause	Fix
`ResourceNotFoundException`	Wrong table name, or **wrong Region**	Check `aws configure get region` = `us-east-1`
`ValidationException: key element does not match the schema`	Key missing, wrong name or wrong type (`S` vs `N`)	Give the exact key attributes with the right types
`ValidationException: … reserved keyword: name`	`name` , `status` , `date` , `data` are reserved words	Use a placeholder: `#n` with `--expression-attribute-names '{"#n":"name"}'`
`ConditionalCheckFailedException`	Your condition was false — this is the safety check working	Read the item and decide what to do
`ProvisionedThroughputExceeded` / throttling	Hot partition key, or provisioned capacity too low	Better key; on-demand mode
Query returns nothing	Partition key value differs (case, spaces)	`get-item` with the exact value first

26 Build it yourself: what you will build

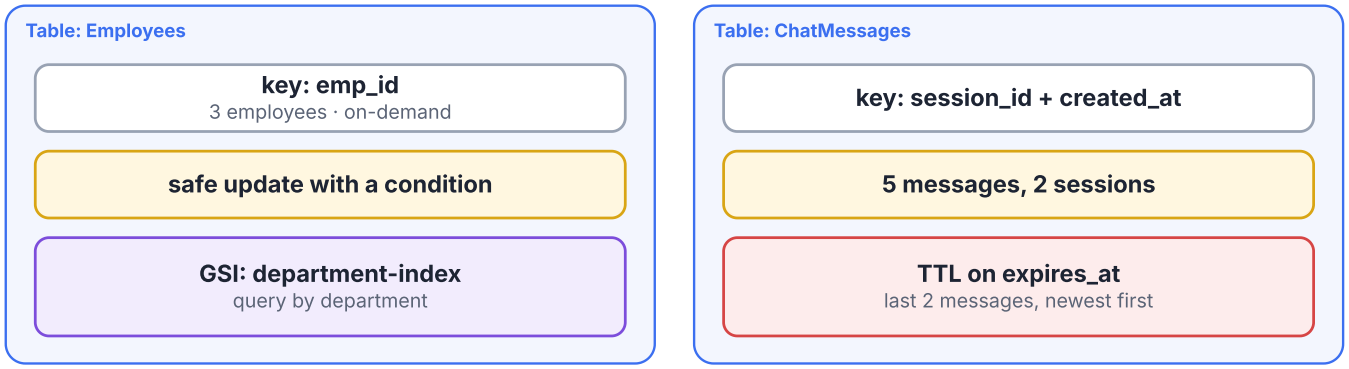


Figure 16. What you will build in us-east-1.

#	You do	Proves
1	Create table Employees	Simple key, on-demand (sections 3, 17)
2	Add 3 employees in one call	Schemaless items, DynamoDB JSON (sections 2, 6)
3	Get one employee; scan all	GetItem vs Scan (section 8)
4	Deduct leave with a condition, then trigger the safety check	Safe updates (section 19)
5	Add a GSI and query by department	Indexes (sections 12, 14)
6	Create ChatMessages with TTL; load the last 2 messages	Composite key, chat memory (sections 4, 9, 21)
7	Delete both tables	No leftovers

> **Cost:** on-demand tables with a few items cost practically nothing and fit in the always-free allowance. Still, do step 7.

27 Before you start

You need an AWS account and the AWS CLI configured (**Module 1, "Before you start"**). In the console, set the Region (top right) to **US East (N. Virginia) us-east-1**. Open the **PyCharm Terminal** (Mac: zsh, Windows: PowerShell) and keep the **same window** open for all steps.

Mac

```
cd ~
mkdir -p ~/aws-dynamodb-lab
cd ~/aws-dynamodb-lab

aws configure set region us-east-1
aws configure get region
```

Windows (PowerShell)

```
cd ~
mkdir ~\aws-dynamodb-lab
cd ~\aws-dynamodb-lab

aws configure set region us-east-1
aws configure get region
```

The CLI wants **DynamoDB JSON** (section 6). To keep commands short and identical on Mac and Windows, each step first writes the JSON into a small file, then the command reads it with `file://name.json`. You can also create these files by hand in PyCharm (right-click the folder → New → File).

28 Build 1 · Create the Employees table

Console

1. Search **DynamoDB** → **Tables** → **Create table**.
2. **Table name:** `Employees`. **Partition key:** `emp_id`, type **String**. Leave **Sort key** empty.
3. **Table settings:** Customize settings → **Capacity mode: On-demand**. Leave everything else as it is.
4. **Create table**. Wait until **Status = Active** (a few seconds).

Command line — Mac

```
aws dynamodb create-table --table-name Employees --attribute-definitions
"AttributeName=emp_id,AttributeType=S" --key-schema "AttributeName=emp_id,KeyType=HASH" --billing-
mode PAY_PER_REQUEST
aws dynamodb wait table-exists --table-name Employees
aws dynamodb describe-table --table-name Employees --query "Table.
[TableName,TableStatus,KeySchema]"
```

Command line — Windows (PowerShell)

```
aws dynamodb create-table --table-name Employees --attribute-definitions
"AttributeName=emp_id,AttributeType=S" --key-schema "AttributeName=emp_id,KeyType=HASH" --billing-
mode PAY_PER_REQUEST
aws dynamodb wait table-exists --table-name Employees
aws dynamodb describe-table --table-name Employees --query "Table.
[TableName,TableStatus,KeySchema]"
```

Only the key attribute is declared (`attribute-definitions`). `name` , `department` and the rest are never declared — they arrive with the items.

29 Build 2 · Add employees

Console

1. Open **Employees** → **Explore table items** → **Create item**.
2. Switch to **JSON view** and turn **off** "View DynamoDB JSON" — now you can type normal JSON.

3. Paste `{"emp_id": "E1001", "name": "Anita", "department": "HR", "city": "Hyderabad", "leave_balance": 18}` → **Create item**.
4. Repeat for Ravi (E1002, Finance, Mumbai, 12) and Sana (E1003, IT, Pune, 20, `"skills": ["python", "aws"]`).

Command line — Mac

```
cat > employees.json <<'EOF'
{
  "Employees": [
    { "PutRequest": { "Item": {
        "emp_id": {"S": "E1001"}, "name": {"S": "Anita"}, "department": {"S": "HR"},
        "city": {"S": "Hyderabad"}, "leave_balance": {"N": "18"} } } },
    { "PutRequest": { "Item": {
        "emp_id": {"S": "E1002"}, "name": {"S": "Ravi"}, "department": {"S": "Finance"},
        "city": {"S": "Mumbai"}, "leave_balance": {"N": "12"} } } },
    { "PutRequest": { "Item": {
        "emp_id": {"S": "E1003"}, "name": {"S": "Sana"}, "department": {"S": "IT"},
        "city": {"S": "Pune"}, "leave_balance": {"N": "20"}, "skills": {"SS": ["python", "aws"]} }
} }
  ]
}
EOF

aws dynamodb batch-write-item --request-items file://employees.json
# "UnprocessedItems": {}  means all 3 were written
```

Command line — Windows (PowerShell)

```
@'
{
  "Employees": [
    { "PutRequest": { "Item": {
        "emp_id": {"S": "E1001"}, "name": {"S": "Anita"}, "department": {"S": "HR"},
        "city": {"S": "Hyderabad"}, "leave_balance": {"N": "18"} } } },
    { "PutRequest": { "Item": {
        "emp_id": {"S": "E1002"}, "name": {"S": "Ravi"}, "department": {"S": "Finance"},
        "city": {"S": "Mumbai"}, "leave_balance": {"N": "12"} } } },
    { "PutRequest": { "Item": {
        "emp_id": {"S": "E1003"}, "name": {"S": "Sana"}, "department": {"S": "IT"},
        "city": {"S": "Pune"}, "leave_balance": {"N": "20"}, "skills": {"SS": ["python", "aws"]} }
} }
  ]
}
'@ | Set-Content -Encoding ascii employees.json

aws dynamodb batch-write-item --request-items file://employees.json
# "UnprocessedItems": {}  means all 3 were written
```

Sana has a `skills` attribute; the others do not. Same table, different shapes — no schema change needed.

30 Build 3 · Read: get and scan

Console

1. **Explore table items** → choose **Query** → `emp_id` = `E1001` → **Run**: one item.
2. Choose **Scan** → **Run**: all three items. Note the count of items **scanned** vs **returned**.
3. Add a filter: attribute `department`, String, equals `IT` → **Run**: still 3 items scanned, 1 returned (section 10).

Command line — Mac

```
cat > key-e1001.json <<'EOF'
{ "emp_id": {"S": "E1001"} }
EOF

# one item by key (fast, cheap)
aws dynamodb get-item --table-name Employees --key file://key-e1001.json

# whole table (fine for 3 items, never in app code)
aws dynamodb scan --table-name Employees --query "Items[].[emp_id.S, department.S,
leave_balance.N]" --output table
```

Command line — Windows (PowerShell)

```
@'
{ "emp_id": {"S": "E1001"} }
'@ | Set-Content -Encoding ascii key-e1001.json

# one item by key (fast, cheap)
aws dynamodb get-item --table-name Employees --key file://key-e1001.json

# whole table (fine for 3 items, never in app code)
aws dynamodb scan --table-name Employees --query "Items[].[emp_id.S, department.S,
leave_balance.N]" --output table
```

31 Build 4 · Update safely

Console

1. Open item **E1001** → change `leave_balance` from 18 to **16** → **Save changes**.
2. The console cannot add conditions — use the command line for the second part.
3. Expected on the command line: the first update prints `"leave_balance": {"N": "16"}`; the second fails with **ConditionalCheckFailedException**.

Command line — Mac

```
cat > deduct-2.json <<'EOF'
{ ":days": {"N": "2"} }
EOF
cat > deduct-100.json <<'EOF'
{ ":days": {"N": "100"} }
EOF

# take 2 days of leave, only if enough is left: 18 -> 16
aws dynamodb update-item --table-name Employees --key file://key-e1001.json --update-expression
"SET leave_balance = leave_balance - :days" --condition-expression "leave_balance >= :days" --
expression-attribute-values file://deduct-2.json --return-values UPDATED_NEW

# try to take 100 days: the condition is false, nothing changes
aws dynamodb update-item --table-name Employees --key file://key-e1001.json --update-expression
"SET leave_balance = leave_balance - :days" --condition-expression "leave_balance >= :days" --
expression-attribute-values file://deduct-100.json
```

Command line — Windows (PowerShell)

```
@'
{ ":days": {"N": "2"} }
'@ | Set-Content -Encoding ascii deduct-2.json
@'
{ ":days": {"N": "100"} }
'@ | Set-Content -Encoding ascii deduct-100.json

# take 2 days of leave, only if enough is left: 18 -> 16
aws dynamodb update-item --table-name Employees --key file://key-e1001.json --update-expression
"SET leave_balance = leave_balance - :days" --condition-expression "leave_balance >= :days" --
expression-attribute-values file://deduct-2.json --return-values UPDATED_NEW

# try to take 100 days: the condition is false, nothing changes
aws dynamodb update-item --table-name Employees --key file://key-e1001.json --update-expression
"SET leave_balance = leave_balance - :days" --condition-expression "leave_balance >= :days" --
expression-attribute-values file://deduct-100.json
```

> The failure in the second command is the point: the database refused to make the balance negative. Your app catches this error and tells the employee "not enough leave".

32 Build 5 · Find by department (GSI)

Console

1. **Employees** → **Indexes** tab → **Create index**.
2. **Partition key:** `department`, String. **Index name:** `department-index`. **Attribute projections:** All → **Create index**.
3. Wait until the index status is **Active**.
4. **Explore table items** → **Query** → select **Index: department-index** → `department` = `HR` → **Run**: Anita.

Command line — Mac

```
cat > gsi.json <<'EOF'
[ { "Create": {
    "IndexName": "department-index",
    "KeySchema": [ { "AttributeName": "department", "KeyType": "HASH" } ],
    "Projection": { "ProjectionType": "ALL" } } } ]
EOF
cat > dept-hr.json <<'EOF'
{ ":d": {"S": "HR"} }
EOF

aws dynamodb update-table --table-name Employees --attribute-definitions
"AttributeName=department,AttributeType=S" --global-secondary-index-updates file://gsi.json

# repeat until it prints ACTIVE (about 1 minute)
aws dynamodb describe-table --table-name Employees --query
"Table.GlobalSecondaryIndexes[0].IndexStatus"

aws dynamodb query --table-name Employees --index-name department-index --key-condition-expression
"department = :d" --expression-attribute-values file://dept-hr.json --query "Items[].[emp_id.S,
name.S]"
```

Command line — Windows (PowerShell)

```
@'
[ { "Create": {
    "IndexName": "department-index",
    "KeySchema": [ { "AttributeName": "department", "KeyType": "HASH" } ],
    "Projection": { "ProjectionType": "ALL" } } } ]
'@ | Set-Content -Encoding ascii gsi.json
@'
{ ":d": {"S": "HR"} }
'@ | Set-Content -Encoding ascii dept-hr.json

aws dynamodb update-table --table-name Employees --attribute-definitions
"AttributeName=department,AttributeType=S" --global-secondary-index-updates file://gsi.json

# repeat until it prints ACTIVE (about 1 minute)
aws dynamodb describe-table --table-name Employees --query
"Table.GlobalSecondaryIndexes[0].IndexStatus"

aws dynamodb query --table-name Employees --index-name department-index --key-condition-expression
"department = :d" --expression-attribute-values file://dept-hr.json --query "Items[].[emp_id.S,
name.S]"
```

33 Build 6 · Chat history table with TTL

Console

1. **Create table** → name `ChatMessages`. **Partition key:** `session_id` (String). **Sort key:** `created_at` (String). Customize settings → **On-demand** → **Create table**.
2. Open it → **Additional settings** tab → **Time to Live (TTL)** → **Turn on** → attribute name `expires_at` → **Turn on TTL**.
3. Add the messages with **Create item** (JSON view, DynamoDB JSON off), e.g. `{"session_id": "S-100", "created_at": "2026-11-01T09:00:01Z", "role": "user", "text": "How many leave days do I have?",`

`"expires_at": 1893456000}`.

4. **Query** → `session_id` = `S-100` → tick **Sort descending** → **Run**: newest first.

Command line — Mac

```
cat > chat.json <<'EOF'
{
  "ChatMessages": [
    { "PutRequest": { "Item": {
        "session_id": {"S": "S-100"}, "created_at": {"S": "2026-11-01T09:00:01Z"},
        "role": {"S": "user"}, "text": {"S": "How many leave days do I have?"},
        "expires_at": {"N": "1893456000"} } } },
    { "PutRequest": { "Item": {
        "session_id": {"S": "S-100"}, "created_at": {"S": "2026-11-01T09:00:03Z"},
        "role": {"S": "assistant"}, "text": {"S": "You have 16 days."},
        "expires_at": {"N": "1893456000"} } } },
    { "PutRequest": { "Item": {
        "session_id": {"S": "S-100"}, "created_at": {"S": "2026-11-01T09:01:10Z"},
        "role": {"S": "user"}, "text": {"S": "Can I carry them over?"},
        "expires_at": {"N": "1893456000"} } } },
    { "PutRequest": { "Item": {
        "session_id": {"S": "S-100"}, "created_at": {"S": "2026-11-01T09:01:12Z"},
        "role": {"S": "assistant"}, "text": {"S": "Up to 10 days."},
        "expires_at": {"N": "1893456000"} } } },
    { "PutRequest": { "Item": {
        "session_id": {"S": "S-200"}, "created_at": {"S": "2026-11-01T10:15:00Z"},
        "role": {"S": "user"}, "text": {"S": "What is the travel policy?"},
        "expires_at": {"N": "1893456000"} } } }
  ]
}
EOF
cat > session-s100.json <<'EOF'
{ ":s": {"S": "S-100"} }
EOF

aws dynamodb create-table --table-name ChatMessages --attribute-definitions
"AttributeName=session_id,AttributeType=S" "AttributeName=created_at,AttributeType=S" --key-schema
"AttributeName=session_id,KeyType=HASH" "AttributeName=created_at,KeyType=RANGE" --billing-mode
PAY_PER_REQUEST
aws dynamodb wait table-exists --table-name ChatMessages
aws dynamodb update-time-to-live --table-name ChatMessages --time-to-live-specification
"Enabled=true,AttributeName=expires_at"

aws dynamodb batch-write-item --request-items file://chat.json

# chat memory: the last 2 messages of session S-100, newest first
aws dynamodb query --table-name ChatMessages --key-condition-expression "session_id = :s" --
expression-attribute-values file://session-s100.json --no-scan-index-forward --limit 2 --query
"Items[].[created_at.S, role.S, text.S]" --output table
```

Command line — Windows (PowerShell)

```
@'
{
  "ChatMessages": [
    { "PutRequest": { "Item": {
        "session_id": {"S": "S-100"}, "created_at": {"S": "2026-11-01T09:00:01Z"},
        "role": {"S": "user"}, "text": {"S": "How many leave days do I have?"},
        "expires_at": {"N": "1893456000"} } } },
    { "PutRequest": { "Item": {
        "session_id": {"S": "S-100"}, "created_at": {"S": "2026-11-01T09:00:03Z"},
        "role": {"S": "assistant"}, "text": {"S": "You have 16 days."},
        "expires_at": {"N": "1893456000"} } } },
    { "PutRequest": { "Item": {
        "session_id": {"S": "S-100"}, "created_at": {"S": "2026-11-01T09:01:10Z"},
        "role": {"S": "user"}, "text": {"S": "Can I carry them over?"},
        "expires_at": {"N": "1893456000"} } } },
    { "PutRequest": { "Item": {
        "session_id": {"S": "S-100"}, "created_at": {"S": "2026-11-01T09:01:12Z"},
        "role": {"S": "assistant"}, "text": {"S": "Up to 10 days."},
        "expires_at": {"N": "1893456000"} } } },
    { "PutRequest": { "Item": {
        "session_id": {"S": "S-200"}, "created_at": {"S": "2026-11-01T10:15:00Z"},
        "role": {"S": "user"}, "text": {"S": "What is the travel policy?"},
        "expires_at": {"N": "1893456000"} } } }
  ]
}
'@ | Set-Content -Encoding ascii chat.json
@'
{ ":s": {"S": "S-100"} }
'@ | Set-Content -Encoding ascii session-s100.json

aws dynamodb create-table --table-name ChatMessages --attribute-definitions
"AttributeName=session_id,AttributeType=S" "AttributeName=created_at,AttributeType=S" --key-schema
"AttributeName=session_id,KeyType=HASH" "AttributeName=created_at,KeyType=RANGE" --billing-mode
PAY_PER_REQUEST
aws dynamodb wait table-exists --table-name ChatMessages
aws dynamodb update-time-to-live --table-name ChatMessages --time-to-live-specification
"Enabled=true,AttributeName=expires_at"

aws dynamodb batch-write-item --request-items file://chat.json

# chat memory: the last 2 messages of session S-100, newest first
aws dynamodb query --table-name ChatMessages --key-condition-expression "session_id = :s" --
expression-attribute-values file://session-s100.json --no-scan-index-forward --limit 2 --query
"Items[].[created_at.S, role.S, text.S]" --output table
```

`expires_at = 1893456000` is 1 Jan 2030 in Unix seconds, so these messages stay. In a real app you write `now + 30 days`, and DynamoDB deletes old chats for you.

The query result is the newest two messages — "Up to 10 days." and "Can I carry them over?" — exactly what an assistant would load as short-term memory before calling the LLM.

34 Build 7 · Clean up (same day)

Console

1. **DynamoDB** → **Tables** → tick `Employees` and `ChatMessages` → **Delete** → type `confirm` → **Delete table**.

Command line — Mac

```
aws dynamodb delete-table --table-name Employees
aws dynamodb delete-table --table-name ChatMessages
aws dynamodb wait table-not-exists --table-name Employees
aws dynamodb wait table-not-exists --table-name ChatMessages
aws dynamodb list-tables

cd ~
```

Command line — Windows (PowerShell)

```
aws dynamodb delete-table --table-name Employees
aws dynamodb delete-table --table-name ChatMessages
aws dynamodb wait table-not-exists --table-name Employees
aws dynamodb wait table-not-exists --table-name ChatMessages
aws dynamodb list-tables

cd ~
```

35 Key takeaways

1. DynamoDB is a serverless NoSQL database: millisecond reads and writes **by key**, at any scale.
2. Only the primary key is fixed. Partition key = where the item lives; sort key = order inside the group.
3. Pick a partition key with many values used evenly — otherwise you get a hot partition.
4. Read with GetItem and Query. Scan reads everything; avoid it in app code. Filters do not lower cost.
5. List your questions first, then design keys and GSIs so each question is one call.
6. Use on-demand capacity, condition expressions for safe updates, and TTL to delete old data for free.
7. For GenAI: DynamoDB is the natural home for chat memory, document status, caches and usage counters.